Formulários para a web

Caderno completo do estudante

Prof. Fabrício Rodrigues - Curso Técnico em Informática

16/10 a 18/12/2026. Nove encontros de duas aulas de 50 minutos. Sem aula em 20/11. Avaliação em 11/12; devolutiva e recuperação em 18/12.

Como usar

Cada aula reúne objetivo, explicação, exemplo, atividade e testes. As aulas de 23/10 em diante têm projetos próprios com três arquivos. Comece pela sua tentativa e consulte o modelo para comparar comportamentos. O envio é simulado e usa dados fictícios.

Baixar e abrir

Baixe o projeto da data, extraia a pasta e abra no VS Code. Use Open with Live Server em index.html. Não há instalação de dependências do projeto. Deixe o VS Code e a extensão disponíveis antes de estudar sem internet.

Registrar e entregar

Na aula de abertura, use a ficha do diagnóstico. Nas demais, entregue a pasta do projeto em ZIP e o TXT do Registro de testes. O professor define a pasta ou o dispositivo de coleta. As páginas não enviam os arquivos automaticamente.

Mapa do percurso

Data	Tema
16/10	O que já sabemos sobre a web?
23/10	Formulários: fundamentos e campos HTML
30/10	Um formulário legível em qualquer tela
06/11	Os dados aparecem na página
13/11	Mensagens que ajudam a corrigir
27/11	O caminho dos dados
04/12	Um formulário que reage ao contexto
11/12	Avaliação prática: inscrição em oficina
18/12	Corrigir, recuperar e concluir

AULA 01 / 16/10/2026

O que já sabemos sobre a web?

Diagnóstico de HTML, CSS e JavaScript e fundamentos de formulários

Prof. Fabrício Rodrigues - Curso Técnico em Informática

Aula de abertura: retome o diagnóstico e os fundamentos conforme os materiais de 16/10 disponíveis no portal.

Diagnóstico como ponto de partida

A atividade inicial serve para orientar as próximas aulas. Registre o que consegue explicar sozinho, o que faz com uma pista e o que ainda precisa estudar. Uma resposta incompleta com raciocínio ajuda o professor a planejar a retomada.

Responda às nove questões antes da conversa coletiva. Na prática, faça pequenas mudanças em uma página conhecida. Depois da tentativa individual, a turma discutirá as estratégias e as dúvidas.

Uma página conhecida, um problema por vez

Na base de diagnóstico, a classe do destaque e os seletores do JavaScript não correspondem ao HTML. Compare os nomes literalmente, leia o console e teste a página após cada alteração. Acrescente um cartão e explique por que escolheu as tags usadas.

Fundamentos: o que é um formulário?

Um formulário é uma interface que permite informar dados e fazer escolhas para realizar uma tarefa. Em vez de apenas ler uma página, a pessoa participa: procura um conteúdo, responde a uma pesquisa ou solicita atendimento.

Antes do código, defina a finalidade, quem vai preencher, quais dados são necessários e qual resultado a pessoa espera. Um bom formulário pede somente o que ajuda a cumprir essa finalidade.

Converse com a turma: qual foi o último formulário que você usou? O que precisou informar e o que aconteceu depois?

Tipos de formulário conforme a finalidade

Os nomes abaixo descrevem usos comuns. Cadastro, acesso e pesquisa não são valores do atributo type do HTML; cada formulário combina campos conforme sua finalidade.

Finalidade	Exemplo de informação e resultado
Cadastro ou inscrição	Informar dados para participar de uma atividade.
Acesso (login)	Apresentar credenciais para solicitar acesso a uma conta.
Pesquisa ou avaliação	Registrar opiniões, respostas ou preferências.
Contato ou suporte	Descrever uma dúvida ou um problema para solicitar atendimento.
Busca ou filtro	Informar um termo ou escolher critérios para encontrar resultados.

Tipos de campo: escolher pelo dado

O controle deve combinar com a informação solicitada. input possui vários tipos definidos pelo atributo type. textarea e select são elementos próprios.

Neste primeiro contato, reconheça as possibilidades. Na prática de hoje, usaremos apenas texto curto para equipamento e texto longo para descrição. Os demais controles serão construídos em 23/10.

Informação desejada	Controle HTML e exemplo
Texto em uma linha	input type="text": código PC-07. Para contato, email; para senha, password.
Descrição em várias linhas	textarea: relato do problema.
Uma opção do grupo	input type="radio" com o mesmo name, ou select: prioridade ou categoria.
Opções independentes	input type="checkbox": sintomas observados.
Data ou quantidade	input type="date" ou type="number", conforme o dado. Um código como PC-07 continua sendo texto.
Escolha de arquivo	input type="file": selecionar um anexo. Selecionar não significa enviar.

Mapa: do preenchimento ao resultado

Pessoa preenche ou escolhe → navegador confere as regras definidas → pessoa solicita a ação → sistema trata os dados → interface informa o resultado.

No projeto da turma, o JavaScript trata a ação no próprio navegador e o envio é simulado. A versão final mostrará um resumo local. Em um sistema conectado, uma requisição pode levar os dados a um servidor, que também deve validá-los e devolver uma resposta.

Preencher, validar, enviar e armazenar são etapas diferentes. Uma mensagem escrita na página só comprova o resultado que o sistema realmente verificou. A escolha de armazenar depende da finalidade e da implementação.

Antes de abrir o código: para um chamado de suporte, qual campo serve para PC-07? Qual serve para explicar o defeito? Que informação seria desnecessária? Justifique suas escolhas.

A ponte para formulários

Um chamado de suporte precisa receber informações que uma página de divulgação apenas exibe. Começaremos com equipamento e descrição do problema. Os dados do exercício são fictícios e ficam no navegador.

O label descreve o campo. Seu atributo for deve apontar para o id do controle. O name será a chave usada na coleta dos valores.

```
<form id="chamado">
  <label for="equipamento">Equipamento</label>
  <input id="equipamento" name="equipamento">
  <label for="descricao">Descrição do problema</label>
  <textarea id="descricao" name="descricao"></textarea>
  <button type="button">Protótipo em construção</button>
</form>
```

Consulte as referências gerais ao final do caderno.

AULA 02 / 23/10/2026

Formulários: fundamentos e campos HTML

Finalidade, estrutura, controles e validação nativa

Prof. Fabrício Rodrigues - Curso Técnico em Informática

Objetivos

• Explicar para que serve um formulário e escolher os dados necessários.

• Associar rótulos e distinguir id, name, type e value.

• Construir escolhas únicas, múltiplas e campos com regras nativas.

Ponto de partida

Retome HTML, CSS e eventos vistos anteriormente. Se o primeiro formulário ficou pendente em 16/10, comece pelo campo de solicitante já fornecido nesta base.

Mapa da aula

Finalidade	Exemplo de uso
Cadastro ou inscrição	Registrar participação em uma atividade.
Busca ou filtro	Encontrar informações.
Contato ou suporte	Descrever uma solicitação.
No projeto da turma	Preparar um chamado fictício; sem envio real.

Especificação dos campos

Campo	Controle	Regra
Solicitante	text	2 a 60 caracteres; obrigatório.
Equipamento	text	2 a 20 caracteres; obrigatório.
Contato	email	Formato e preenchimento obrigatório.
Categoria	select	Opção inicial vazia; escolha obrigatória.
Prioridade	radio	Normal ou Alta; uma escolha obrigatória.
Descrição	textarea	20 a 500 caracteres; obrigatório.
Sintomas	checkbox	Lentidão e Mensagem de erro; opcionais.

Fundamentos antes das tags

Um formulário permite informar dados e fazer escolhas para realizar uma tarefa. A finalidade orienta os campos: num chamado precisamos identificar o equipamento, classificar o problema e descrevê-lo.

Cadastro, login, pesquisa, contato e busca são finalidades. text, email e radio são tipos de controle; não são categorias de formulário. Nesta aula construiremos a estrutura HTML. CSS e JavaScript já fornecidos apenas mantêm o exemplo legível e a simulação local.

O papel de form, label e dos atributos

form reúne os controles da tarefa. label informa o que preencher. O for do label deve corresponder ao id do campo. Clique no rótulo para verificar a associação. Um placeholder é só um exemplo e não substitui o rótulo.

id identifica o elemento no documento. name identifica o dado na coleta. type escolhe o comportamento. value representa o valor do controle. Dois campos podem participar do mesmo grupo pelo name, mas seus ids devem ser únicos. Um código como PC-07 é texto, não uma quantidade.

```
<label for="contato">E-mail fictício (obrigatório)</label>
<input id="contato" name="contato" type="email" required>
```

Uma lista de opções com select

select reúne opções conhecidas; cada option tem um valor de dados e um texto visível. A opção inicial com value vazio permite que required peça uma escolha.

No chamado, use hardware, software, rede e outro. Nesta aula, escolher outro ainda não abre um campo adicional. Esse comportamento será construído em 04/12.

```
<label for="categoria">Categoria (obrigatório)</label>
<select id="categoria" name="categoria" required>
  <option value="">Escolha uma categoria</option>
  <option value="hardware">Hardware</option>
  <option value="software">Software</option>
  <option value="rede">Rede</option>
  <option value="outro">Outro</option>
</select>
```

Escolha única com radio

Radios com o mesmo name formam um grupo de escolha única. Values diferentes distinguem as respostas. fieldset reúne o grupo e legend explica a pergunta.

Escolha prioridade normal ou alta. O required em um radio do grupo exige que uma opção daquele grupo seja escolhida. Não crie names diferentes para cada alternativa.

```
<fieldset>
  <legend>Prioridade (obrigatória)</legend>
  <label><input type="radio" name="prioridade"
    value="normal" required> Normal</label>
  <label><input type="radio" name="prioridade"
    value="alta"> Alta</label>
</fieldset>
```

Escolhas independentes com checkbox

Checkboxes permitem marcar nenhuma, uma ou várias opções independentes. Use sintomas opcionais: Lentidão e Mensagem de erro.

Os dois controles podem ter o mesmo name e values diferentes. Na coleta com JavaScript, usaremos getAll para recuperar todas as escolhas, em 06/11. required em um checkbox exige aquele checkbox específico; não significa “pelo menos um do conjunto”.

```
<fieldset>
  <legend>Sintomas (opcionais)</legend>
  <label><input type="checkbox" name="sintomas"
    value="Lentidão"> Lentidão</label>
  <label><input type="checkbox" name="sintomas"
    value="Mensagem de erro"> Mensagem de erro</label>
</fieldset>
```

Texto longo e regras nativas

textarea recebe a descrição. A instrução informa de 20 a 500 caracteres, e aria-describedby associa a ajuda ao campo. required, minlength e maxlength são regras que o navegador pode conferir.

type=email verifica um formato, sem comprovar a existência da caixa de e-mail. As mensagens variam por navegador. A versão desta aula testa as regras HTML; comparação entre e-mails e texto formado só por espaços ficam para 13/11.

```
<label for="descricao">Descrição (obrigatória)</label>
<p id="ajuda-descricao">Use de 20 a 500 caracteres.</p>
<textarea id="descricao" name="descricao" rows="5"
  required minlength="20" maxlength="500"
  aria-describedby="ajuda-descricao"></textarea>
```

Prática no VS Code

Baixe o projeto específico desta aula. Extraia o ZIP, abra a pasta no VS Code e execute index.html com Live Server. Os três arquivos são index.html, style.css e script.js.

1. Baixar o projeto desta aula, extrair os três arquivos e abrir a pasta no VS Code. Executar index.html com Live Server.

2. Usar o campo de solicitante como modelo. Acrescentar equipamento (texto, 2 a 20 caracteres), contato (email), categoria (select), prioridade (radio), descrição (20 a 500 caracteres) e os dois sintomas opcionais (checkbox).

3. Associar rótulos, usar names coerentes e deixar visível o que é obrigatório. Manter o script de simulação fornecido.

4. Salvar, testar os casos e explicar a diferença entre id e name usando um campo do seu projeto.

Casos de teste

Compare o esperado com o observado. Registre falhas e correções no TXT da aula.

Caso	Resultado esperado
Campos obrigatórios vazios	O navegador impede a simulação.

Caso	Resultado esperado
Contato sem formato de e-mail	O campo de contato fica inválido.
Descrição digitada com menos de 20 caracteres	O navegador pede completar o texto.
Clique no rótulo do equipamento	O foco vai para o campo correspondente.
Marcar duas prioridades em sequência	Só a última permanece marcada.
Marcar os dois sintomas	As duas opções permanecem marcadas.
Preenchimento válido	Mensagem local; nenhuma navegação de envio.

Erros que merecem atenção

• name não substitui id na associação com label.

• Um select com opção inicial já válida não testa a escolha obrigatória.

• Não acrescente novalidate: queremos observar a validação HTML.

Conferência

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Qual é o papel de name? Qual teste mostrou que seu formulário estava funcionando?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Da aula 02 em diante: salve os três arquivos em ZIP, abra o Registro de testes no portal, informe nome e turma e baixe o TXT. Entregue ZIP e TXT ao professor na aula. Não há envio nem salvamento automático.

Referências com links

MDN: seu primeiro formulário

MDN: controles nativos de formulário (em inglês)

W3C WAI: rótulos dos controles (em inglês)

MDN: tipos de input (em inglês)

W3C WAI: agrupamento de controles (em inglês)

MDN: validação de formulários

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 03 / 30/10/2026

Um formulário legível em qualquer tela

CSS, responsividade, instruções e navegação pelo teclado

Prof. Fabrício Rodrigues - Curso Técnico em Informática

Objetivos

• Organizar os campos em uma e duas colunas sem rolagem horizontal.

• Manter rótulos, instruções e foco legíveis.

• Testar teclado e ampliação sem depender somente da aparência.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Mapa da aula

Camada	Responsabilidade
HTML	Preserva rótulos, grupos e regras.
CSS	Organiza largura, espaçamento e foco.
Teclado	Permite percorrer e acionar os controles.
Conferência	Telas estreitas, ampliação e sequência de foco.

Layout começa pela tela estreita

Abra a base com o HTML completo da aula anterior. A regra global box-sizing inclui borda e preenchimento na largura calculada. Evite campos com largura fixa maior que a tela.

Uma grade com gap organiza os campos em uma coluna. O código fornecido já mantém inputs com largura de 100% e min-width:0. O objetivo da aula é completar o layout sem alterar os names e as regras HTML.

```
* { box-sizing: border-box; }
.campos {
  display: grid;
  gap: 1.2rem;
}
.campo { min-width: 0; }
```

Duas colunas quando há espaço

A media query entra a partir de 48rem. Até esse limite, a grade continua em uma coluna. O campo de descrição ocupa a linha inteira por meio da classe largo.

O ponto de mudança pertence a este projeto. Ele não define um tipo fixo de aparelho: observe quando os campos deixam de caber com conforto.

```
@media (min-width: 48rem) {
  .campos { grid-template-columns: 1fr 1fr; }
  .largo { grid-column: 1 / -1; }
}
```

Foco visível e ordem de leitura

Tab percorre os controles e Shift+Tab retorna. A ordem do HTML deve acompanhar a ordem de leitura; não use tabindex positivo para reorganizar o formulário.

focus-visible mostra uma indicação clara para navegação. Não remova o contorno sem oferecer indicação equivalente. Rótulos e instruções continuam visíveis durante o preenchimento.

```
:focus-visible {
  outline: 3px solid #6a8f32;
  outline-offset: 4px;
}
```

Teste da experiência

Teste aproximadamente 360px, 768px e 1280px, além da ampliação do navegador em 200%. Compare largura, tamanho dos textos, quebra das instruções e acesso aos botões.

No modo responsivo do navegador, altere a largura e observe o resultado. Medir apenas a quantidade de colunas não comprova que o formulário ficou utilizável.

Prática no VS Code

Baixe o projeto específico desta aula. Extraia o ZIP, abra a pasta no VS Code e execute index.html com Live Server. Os três arquivos são index.html, style.css e script.js.

1. Abrir a base desta aula ou continuar o HTML concluído no encontro anterior.

2. Completar a media query para duas colunas e fazer a descrição ocupar a largura da grade.

3. Acrescentar foco visível e revisar espaçamentos dos grupos e das instruções.

4. Pedir a um colega que percorra o formulário pelo teclado e repetir os testes nas larguras indicadas.

Casos de teste

Compare o esperado com o observado. Registre falhas e correções no TXT da aula.

Caso	Resultado esperado
Tela de aproximadamente 360px	Uma coluna; controles sem corte lateral.
Tela larga a partir de 48rem	Duas colunas; descrição ocupa a linha.
Tab e Shift+Tab	Foco visível, sequência compreensível.
Ampliação de 200%	Conteúdo legível e controles alcançáveis.
Texto longo na instrução	Quebra de linha sem sobrepor outro campo.
Preenchimento inválido	Regras HTML da aula anterior continuam ativas.

Erros que merecem atenção

• Trocar a aparência de um campo não deve retirar seu rótulo.

• Uma largura fixa de 900px pode quebrar a versão estreita.

• Não usar cor como única indicação de erro ou obrigatoriedade.

Conferência

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Qual decisão ajudou a tela estreita? Como você conferiu o foco?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Da aula 02 em diante: salve os três arquivos em ZIP, abra o Registro de testes no portal, informe nome e turma e baixe o TXT. Entregue ZIP e TXT ao professor na aula. Não há envio nem salvamento automático.

Referências com links

MDN: estilização de formulários (em inglês)

MDN: fundamentos de CSS Grid

W3C WAI: rótulos dos controles (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 04 / 06/11/2026

Os dados aparecem na página

Evento submit, FormData e resumo local com JavaScript

Prof. Fabrício Rodrigues - Curso Técnico em Informática

Objetivos

• Escutar submit no formulário e impedir a navegação na simulação.

• Coletar campos por name, preservando múltiplos checkboxes.

• Montar um resumo com elementos do DOM e textContent.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Mapa da aula

Passo	O que acontece
Preencher	A pessoa informa dados fictícios.
Validar	O navegador aplica as regras HTML.
submit	O evento chega ao formulário válido.
JavaScript	Impede a navegação e apresenta o resumo local.

O evento pertence ao formulário

Usar submit permite tratar o botão e a submissão pelo teclado. A validação HTML interativa acontece antes do submit: campos inválidos podem impedir que esse evento aconteça.

preventDefault interrompe a ação padrão nesta demonstração. Ele não valida, não envia para um servidor e não grava os dados. A base já oferece um bloqueio de navegação; substitua a mensagem pelo processamento do resumo.

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  const dados = new FormData(form);
  mostrarResumo(dados);
});
```

Coleta por name

new FormData(form) monta os pares de dados dos controles participantes. get recupera um valor; getAll recupera todos os valores com o mesmo name. Checkboxes desmarcados não entram na coleta.

Um campo com id e sem name continua aparecendo na tela, mas não entra como par nome/valor na coleta. Os sintomas selecionados precisam aparecer juntos no resumo.

```
const dados = new FormData(form);
const contato = dados.get('contato');
const sintomas = dados.getAll('sintomas');
// getAll preserva várias opções com o mesmo name.
```

Resumo como conteúdo, sem interpretar HTML

Crie dt e dd para cada item da lista de descrição. textContent apresenta o valor como texto. Não use innerHTML para inserir o que a pessoa digitou.

Limpe o resumo anterior antes de montar o novo. Depois torne a seção visível e leve o foco ao título com tabindex=-1. O resumo corresponde à última simulação, não a um registro persistente.

```
function adicionarLinha(titulo, valor) {
  const dt = document.createElement('dt');
  const dd = document.createElement('dd');
  dt.textContent = titulo;
  dd.textContent = String(valor);
  resumo.append(dt, dd);
}
```

Estado do formulário depois da tentativa

Uma falha de validação deve manter os valores para correção. Um novo envio deve substituir o resumo anterior, sem duplicar linhas.

O botão Novo preenchimento é type=button. Ele pode chamar reset, ocultar o resumo e devolver o foco ao primeiro campo. Avise que o projeto perde os dados quando a página é recarregada.

Prática no VS Code

Baixe o projeto específico desta aula. Extraia o ZIP, abra a pasta no VS Code e execute index.html com Live Server. Os três arquivos são index.html, style.css e script.js.

1. Abrir a base da aula com HTML e CSS concluídos.

2. Substituir o tratamento de submit pela coleta com FormData e pela chamada que monta o resumo.

3. Mostrar solicitante, equipamento, contato, categoria, prioridade, descrição e todos os sintomas selecionados.

4. Acrescentar a limpeza do resumo em uma nova simulação e o botão de novo preenchimento.

Casos de teste

Compare o esperado com o observado. Registre falhas e correções no TXT da aula.

Caso	Resultado esperado
Campo obrigatório vazio	O resumo não aparece.
Preenchimento válido	Resumo local com os valores informados.
Dois sintomas marcados	Ambos aparecem no resumo.
Nenhum sintoma marcado	Mensagem de nenhuma opção selecionada.
Texto com exemplo na descrição	As marcas aparecem como texto, sem formatação HTML.

Caso	Resultado esperado
Duas simulações válidas	O resumo mais recente substitui o anterior.
Novo preenchimento	Campos reiniciados e foco no primeiro campo.

Erros que merecem atenção

• Escutar somente click do botão deixa de representar a tarefa do formulário.

• get recupera um único valor; sintomas exigem getAll.

• Resumo na tela não significa que o chamado foi enviado ou salvo.

Conferência

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Como o name chega ao FormData? Por que usar textContent no resumo?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Da aula 02 em diante: salve os três arquivos em ZIP, abra o Registro de testes no portal, informe nome e turma e baixe o TXT. Entregue ZIP e TXT ao professor na aula. Não há envio nem salvamento automático.

Referências com links

MDN: evento submit (em inglês)

MDN: validação de formulários

MDN: uso de FormData (em inglês)

MDN: textContent (em inglês)

MDN: elemento button (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 05 / 13/11/2026

Mensagens que ajudam a corrigir

Regras complementares, validade personalizada e casos de teste

Prof. Fabrício Rodrigues - Curso Técnico em Informática

Objetivos

• Comparar o contato com sua confirmação sem perder as regras nativas.

• Criar e limpar mensagens com setCustomValidity.

• Registrar casos válidos, inválidos e limites da descrição.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Mapa da aula

Regra	Camada do protótipo
Obrigatoriedade e formato	Atributos HTML.
Confirmação de contato	JavaScript compara dois campos.
Descrição útil	JavaScript confere o texto sem espaços nas pontas.
Aplicação real	Servidor precisaria conferir as regras novamente.

Comparação de dois campos

A base acrescenta o campo de confirmação com type=email e required. Isso confere cada formato, mas não prova que os dois valores são iguais.

A função compara os valores e define uma mensagem quando diferem. String vazia limpa a validade personalizada. Escute input nos dois campos, pois qualquer um pode mudar depois.

```
function conferirContato() {
  const diferente = confirmar.value !== contato.value;
  confirmar.setCustomValidity(
    confirmar.value && diferente ? 'Repita o mesmo e-mail.' : ''
  );
}
contato.addEventListener('input', conferirContato);
confirmar.addEventListener('input', conferirContato);
```

Descrição e limites do teste

minlength e maxlength tratam o comprimento do valor. Nesta aula acrescentamos a regra de pelo menos 20 caracteres depois de trim, para não aceitar uma descrição formada só por espaços.

A função limpa a mensagem quando o texto passa. Teste digitando no campo; restrições de comprimento nativas têm particularidades quando um script atribui o valor diretamente. Não altere o valor do aluno em silêncio.

```
function conferirDescricao() {
  const curta = descricao.value.length > 0
    && descricao.value.trim().length < 20;
  descricao.setCustomValidity(
    curta ? 'Descreva o problema com pelo menos 20 caracteres.' : ''
  );
}
descricao.addEventListener('input', conferirDescricao);
```

Nova tentativa sem mensagens antigas

Antes de montar o resumo, atualize as verificações e consulte reportValidity. Os listeners de input são essenciais para limpar um erro anterior que bloquearia a próxima submissão.

Ao reiniciar, faça reset e limpe a validade personalizada. Mostrar um aviso não deve apagar os campos nem transferir a culpa para a pessoa. A mensagem precisa indicar como corrigir.

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  conferirContato();
  conferirDescricao();
  if (!form.reportValidity()) return;
  mostrarResumo(new FormData(form));
});
```

Prática no VS Code

Baixe o projeto específico desta aula. Extraia o ZIP, abra a pasta no VS Code e execute index.html com Live Server. Os três arquivos são index.html, style.css e script.js.

1. Abrir a base e localizar os campos contato, confirmar e descricao.

2. Implementar a comparação e registrar input nos dois e-mails.

3. Acrescentar a regra da descrição com trim e limpar as mensagens quando o valor for corrigido.

4. Repetir a tentativa após cada correção e registrar os limites e o reinício.

Casos de teste

Compare o esperado com o observado. Registre falhas e correções no TXT da aula.

Caso	Resultado esperado
E-mails diferentes	Mensagem de repetir o mesmo contato.
Corrigir o segundo e-mail	A mensagem antiga é limpa.
Mudar o primeiro depois da confirmação	A divergência volta a ser identificada.
Descrição formada por espaços	A regra complementar bloqueia a simulação.
Descrição digitada com 19 caracteres úteis	A simulação fica bloqueada.
Descrição digitada com 20 caracteres úteis	A regra de tamanho permite continuar.

Caso	Resultado esperado
Novo preenchimento após erro	Não permanece mensagem personalizada antiga.

Erros que merecem atenção

• Mensagem personalizada sem limpeza mantém o campo inválido.

• Comparar apenas no click não cobre todas as formas de tentativa.

• Validação no navegador não substitui a conferência em um servidor real.

Conferência

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Quando sua mensagem é limpa? Que teste demonstrou a correção do erro anterior?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Da aula 02 em diante: salve os três arquivos em ZIP, abra o Registro de testes no portal, informe nome e turma e baixe o TXT. Entregue ZIP e TXT ao professor na aula. Não há envio nem salvamento automático.

Referências com links

MDN: setCustomValidity (em inglês)

MDN: validação de formulários

W3C WAI: notificações e erros (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 06 / 27/11/2026

O caminho dos dados

GET, POST, HTTPS e contrato do projeto integrador

Prof. Fabrício Rodrigues - Curso Técnico em Informática

Objetivos

• Distinguir uma simulação local de uma requisição a um serviço.

• Comparar GET e POST e explicar o papel do HTTPS.

• Documentar campos e regras de um serviço futuro sem inventar um backend.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Mapa da aula

Situação	Representação
Busca de dados com GET	Parâmetros podem aparecer na URL.
Envio com POST	Dados de formulário podem seguir no corpo.
HTTPS	Protege o tráfego em trânsito.
Projeto no GitHub Pages	Site estático; não implementa recebimento de chamados.

Os limites do protótipo

Até aqui o JavaScript trabalha somente na página. Mostrar um resumo não faz uma requisição de envio. O GitHub Pages publica os arquivos estáticos do curso; não executa um serviço próprio para receber chamados.

Uma aplicação real precisaria de uma rota de servidor e de regras de recebimento. Nesta aula documentaremos esse contrato e compararemos representações, sem enviar dados a um endereço externo.

GET e os parâmetros da URL

GET costuma ser usado para recuperar informações. Em um formulário com method=get, os pares de dados podem compor a query da URL. Eles podem aparecer no histórico e ser compartilhados junto com o endereço.

O exemplo gera uma string com URLSearchParams e a mostra como texto. exemplo.test é um domínio de exemplo; o projeto não navega para ele. Observe como nomes, espaços e caracteres são representados.

```
const dados = new FormData(form);
const parametros = new URLSearchParams(dados);
const exemplo = 'https://exemplo.test/chamados?'
  + parametros.toString();
// Apenas exibir o texto. Não navegar para esse endereço.
```

POST, HTTPS e conferência no servidor

Em um formulário HTML com method=post, os dados podem seguir no corpo da requisição. POST sozinho não criptografa dados. HTTPS protege o tráfego em trânsito. Não coloque informações sensíveis em uma URL por achar que o método resolveu toda a proteção.

O servidor deve conferir formato, tamanho, escolhas permitidas e demais regras. O usuário pode alterar a página ou enviar outra requisição. A validação HTML e o JavaScript servem à experiência local e precisam de conferência no serviço real.

Contrato do integrador

Documente no HTML um bloco de contrato: finalidade, método escolhido, rota proposta, campos e regras a aplicar no servidor. Identifique claramente que a rota não está implementada.

A base exibe exemplos de URL GET e de um corpo para POST depois da simulação. São representações de estudo. Complete o contrato e confronte seus nomes de campos com o que FormData realmente coleta.

```
<!-- Exemplo de contrato para uma aplicação futura.
Método: POST. Rota proposta: /chamados.
O servidor deve validar os campos novamente.
Este projeto estático não implementa essa rota.
-->
```

Prática no VS Code

Baixe o projeto específico desta aula. Extraia o ZIP, abra a pasta no VS Code e execute index.html com Live Server. Os três arquivos são index.html, style.css e script.js.

1. Abrir a base da aula e observar o resumo e as representações de estudo.

2. Comparar onde os valores apareceriam em GET e POST e explicar por que não houve envio real.

3. Completar o contrato no HTML: finalidade, POST, rota proposta /chamados, campos, regras e indicação de serviço ainda não implementado.

4. Conferir se os names coletados correspondem ao contrato e registrar o escopo do integrador.

Casos de teste

Compare o esperado com o observado. Registre falhas e correções no TXT da aula.

Caso	Resultado esperado
Simulação com dados fictícios	Resumo e representações locais, sem envio.
Espaços no nome	URL de exemplo com parâmetros codificados.
Dois sintomas selecionados	Representação de POST preserva a lista.
Contrato sem indicar implementação futura	Revisar para não prometer serviço existente.

Caso	Resultado esperado
POST sem HTTPS no argumento	Explicar que POST não oferece criptografia.
Campo sem name	Identificar a ausência na coleta e corrigir o contrato.

Erros que merecem atenção

• O corpo de POST não é uma garantia de criptografia.

• Não afirmar que o Pages recebeu, validou ou armazenou o chamado.

• Um contrato é uma proposta; não é prova de que o endpoint existe.

Conferência

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Qual parte do seu projeto roda no navegador? O que ainda faltaria num serviço real?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Da aula 02 em diante: salve os três arquivos em ZIP, abra o Registro de testes no portal, informe nome e turma e baixe o TXT. Entregue ZIP e TXT ao professor na aula. Não há envio nem salvamento automático.

Referências com links

GitHub: funcionamento do Pages

MDN: envio de dados (em inglês)

MDN: HTTPS (em inglês)

MDN: uso de FormData (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 07 / 04/12/2026

Um formulário que reage ao contexto

Campos condicionais, estado e organização das funções

Prof. Fabrício Rodrigues - Curso Técnico em Informática

Objetivos

• Mostrar um campo adicional somente quando sua opção for escolhida.

• Coordenar hidden, disabled e required.

• Repetir a coleta e o reinício sem valores de uma opção antiga.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Mapa da aula

Categoria	Estado do campo adicional
Outra opção não escolhida	Bloco oculto, campo desabilitado e não obrigatório.
Outro escolhido	Bloco visível, campo habilitado e obrigatório.
Retorno a uma categoria comum	Limpar o valor antigo e desativar o campo.
Coleta	Somente os campos participantes entram no FormData.

A interface responde a uma escolha

O HTML da base já contém o bloco de outra categoria e um input inicialmente disabled. A função de estado deve ler o select e atualizar o conjunto de propriedades.

Registre change no select e chame a função ao iniciar a página. Organize a regra em uma função, para também aplicá-la depois de reset. Não espalhe a mesma decisão em vários handlers.

```
function atualizarOutro() {
  const ativo = categoria.value === 'outro';
  blocoOutro.hidden = !ativo;
  outro.disabled = !ativo;
  outro.required = ativo;
  if (!ativo) outro.value = '';
}
categoria.addEventListener('change', atualizarOutro);
atualizarOutro();
```

Esconder não desativa o controle

hidden altera a apresentação do bloco. Um campo obrigatório somente escondido ainda pode interferir na validação. disabled retira o controle da validação e da coleta normal de dados.

required deve ficar ativo apenas quando o campo é necessário. Ao desativar a opção, o projeto apaga o valor antigo para não sugerir um dado que deixou de pertencer à escolha atual.

```
// hidden controla a apresentação do bloco.
// disabled retira o campo da validação e do FormData.
// required pede um valor quando o campo está ativo.
```

Conferência do fluxo inteiro

Teste entrar em Outro, tentar sem preencher, informar um detalhe e gerar o resumo. Depois retorne a Hardware e confira que o detalhe não aparece na coleta.

Use Novo preenchimento para conferir reset e reaplicação do estado. Revise a comparação dos e-mails e as mensagens: acrescentar uma condição não deve quebrar as regras anteriores.

Prática no VS Code

Baixe o projeto específico desta aula. Extraia o ZIP, abra a pasta no VS Code e execute index.html com Live Server. Os três arquivos são index.html, style.css e script.js.

1. Abrir a base e localizar o select, o bloco oculto e o campo adicional.

2. Criar a função de estado, escutar change e aplicá-la na inicialização.

3. Manter o campo adicional obrigatório apenas em Outro e incluir seu valor no resumo somente quando coletado.

4. Testar todas as transições e executar novamente os casos de validação das aulas anteriores.

Casos de teste

Compare o esperado com o observado. Registre falhas e correções no TXT da aula.

Caso	Resultado esperado
Abertura com categoria vazia	Campo adicional oculto e desabilitado.
Escolher Outro sem detalhar	Campo visível e obrigatório.
Outro com detalhe e dados válidos	Resumo inclui a outra categoria.
Trocar de Outro para Hardware	Bloco oculto e valor antigo limpo.
Gerar resumo após sair de Outro	Não inclui outra_categoria.
Novo preenchimento	Estado inicial reaplicado.

Erros que merecem atenção

• hidden sozinho não expressa a participação do campo.

• Não deixar required ativo num campo desabilitado por lógica incompleta.

• reset não substitui a reaplicação explícita da regra de estado.

Conferência

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Que diferença existe entre esconder um campo e desabilitá-lo?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Da aula 02 em diante: salve os três arquivos em ZIP, abra o Registro de testes no portal, informe nome e turma e baixe o TXT. Entregue ZIP e TXT ao professor na aula. Não há envio nem salvamento automático.

Referências com links

MDN: atributo disabled (em inglês)

MDN: evento submit (em inglês)

MDN: uso de FormData (em inglês)

MDN: setCustomValidity (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 08 / 11/12/2026

Avaliação prática: inscrição em oficina

Aplicação individual dos conhecimentos de formulários

Prof. Fabrício Rodrigues - Curso Técnico em Informática

Objetivos

• Construir um formulário para um contexto diferente do chamado.

• Demonstrar estrutura, responsividade, coleta e validação.

• Registrar testes e explicar uma decisão de implementação.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Mapa da aula

Critério	Valor proposto
HTML e controles adequados	2 pontos.
CSS, rótulos e teclado	2 pontos.
Coleta e resumo local	2 pontos.
Validação e campo condicional	2 pontos.
Testes e explicação individual	2 pontos.

Situação da avaliação

Uma oficina de informática precisa de um protótipo de inscrição. Use somente dados fictícios e mantenha o envio simulado. O professor explicará as regras de consulta e os critérios antes do início.

O participante deve escolher uma oficina (Redes, Hardware, Web ou Outra oficina) e um turno (Manhã ou Tarde). Materiais que levará são opcionais: Caderno e Notebook.

Especificação dos campos

Participante fictício: texto de 2 a 60 caracteres. Contato e confirmação: formato de e-mail, obrigatórios e iguais. Oficina: select obrigatório com opção inicial vazia. Turno: uma opção obrigatória em radio.

Objetivo da participação: de 20 a 500 caracteres úteis. Ao escolher Outra oficina, mostrar Tema sugerido, habilitá-lo e torná-lo obrigatório. Ao sair dessa escolha, limpar e desabilitar esse campo.

Comportamento e entrega

Mostrar um resumo local com os dados da inscrição, preservando os dois materiais opcionais se marcados. Tratar valores como texto. Não enviar nem gravar em servidor.

O formulário deve funcionar em tela estreita e pelo teclado. Manter valores após uma tentativa inválida. O reinício precisa limpar as mensagens e o estado condicional. Entregar os três arquivos em ZIP e o registro TXT.

Prática no VS Code

Baixe o projeto específico desta aula. Extraia o ZIP, abra a pasta no VS Code e execute index.html com Live Server. Os três arquivos são index.html, style.css e script.js.

1. Ler os requisitos e os critérios com o professor. Baixar a base da avaliação.

2. Construir individualmente o HTML, completar CSS e JavaScript conforme a especificação.

3. Realizar os testes da tabela e registrar os resultados reais, inclusive o que ainda falhou.

4. Entregar ZIP e TXT e explicar um campo, uma regra e um teste ao professor.

Casos de teste

Compare o esperado com o observado. Registre falhas e correções no TXT da aula.

Caso	Resultado esperado
Obrigatórios vazios	Bloqueio da tentativa.
E-mails diferentes e depois corrigidos	Erro comunicado e mensagem limpa ao corrigir.
Outra oficina sem tema	Tema obrigatório.
Trocar de Outra oficina para Web	Tema deixa a validação e a coleta.
Caderno e Notebook marcados	Resumo preserva os dois materiais.
Texto com sinais de tag no objetivo	Aparece como texto literal.
Tela estreita e teclado	Campos alcançáveis e foco visível.
Novo preenchimento	Regras e estado voltam à condição inicial.

Erros que merecem atenção

• A entrega visual não substitui a explicação do código.

• Não importar o resumo completo do projeto de chamado sem adaptar os names.

• O registro de teste precisa distinguir previsão e resultado observado.

Conferência

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Qual requisito você demonstrou com maior autonomia? Qual precisa corrigir?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Da aula 02 em diante: salve os três arquivos em ZIP, abra o Registro de testes no portal, informe nome e turma e baixe o TXT. Entregue ZIP e TXT ao professor na aula. Não há envio nem salvamento automático.

Referências com links

MDN: seu primeiro formulário

MDN: controles nativos de formulário (em inglês)

MDN: validação de formulários

MDN: atributo disabled (em inglês)

MDN: uso de FormData (em inglês)

W3C WAI: rótulos dos controles (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 09 / 18/12/2026

Corrigir, recuperar e concluir

Devolutiva, depuração e nova evidência de aprendizagem

Prof. Fabrício Rodrigues - Curso Técnico em Informática

Objetivos

• Interpretar a devolutiva com base em critérios e testes.

• Localizar e corrigir defeitos de rótulo, coleta e estado.

• Demonstrar novamente os critérios pendentes.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Mapa da aula

Etapa	Evidência
Devolutiva	Critério, resultado observado e ponto a corrigir.
Correção	Hipótese, alteração e novo teste.
Recuperação	Tarefa equivalente do critério pendente.
Fechamento	Explicação individual e versão corrigida.

Devolutiva sem presumir os resultados

O professor devolve a avaliação real com os critérios e as evidências observadas. Não existe uma nota antecipada neste material. Escolha o que precisa retomar a partir dessa devolutiva.

O projeto de recuperação contém defeitos intencionais para investigar. Quem já demonstrou os critérios pode explicar os defeitos e ampliar os testes, conforme orientação do professor.

Investigação de rótulos e coleta

Clique no rótulo de contato e observe o foco. Compare for com o id. Depois confira se o campo possui name e se a coleta usa o formulário correto.

Faça uma mudança por vez e repita o mesmo teste. Um formulário vazio de dados no resumo pode indicar um problema de coleta, mesmo que os campos tenham sido preenchidos.

```
const dados = new FormData(form);
const contato = dados.get('contato');
const sintomas = dados.getAll('sintomas');
// getAll preserva várias opções com o mesmo name.
```

Estado e nova tentativa

Alterne a opção Outra oficina e confira visibilidade, obrigatoriedade e participação do tema sugerido. Compare as propriedades com a tabela de estados estudada.

Documente o problema, a hipótese, a alteração feita e o resultado depois da correção. Use uma tarefa equivalente para demonstrar os critérios ainda pendentes; siga as regras de recuperação da turma.

```
// hidden controla a apresentação do bloco.
// disabled retira o campo da validação e do FormData.
// required pede um valor quando o campo está ativo.
```

Prática no VS Code

Baixe o projeto específico desta aula. Extraia o ZIP, abra a pasta no VS Code e execute index.html com Live Server. Os três arquivos são index.html, style.css e script.js.

1. Ler a devolutiva e escolher os critérios a retomar com o professor.

2. Abrir a base de recuperação e investigar os defeitos de rótulo, name, coleta e campo condicional.

3. Corrigir uma hipótese por vez e repetir os testes.

4. Entregar a versão corrigida com o registro do progresso e explicar uma correção sem ler a solução.

Casos de teste

Compare o esperado com o observado. Registre falhas e correções no TXT da aula.

Caso	Resultado esperado após a correção
Clicar no rótulo de contato	O foco chega ao contato.
Preencher contato e gerar resumo	O valor informado aparece no resumo.
Escolher Outra oficina	Tema fica habilitado e obrigatório.
Sair de Outra oficina	Tema fica oculto e fora da coleta.
Corrigir um valor inválido	A próxima tentativa pode ocorrer.
Repetir o caso da avaliação	A evidência do critério pendente fica registrada.

Erros que merecem atenção

• Trocar vários trechos ao mesmo tempo dificulta identificar a causa.

• Uma mensagem de sucesso sozinha não comprova o conteúdo do resumo.

• Recuperação deve produzir nova evidência do critério, conforme as regras da turma.

Conferência

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Qual era a causa do erro, como você a corrigiu e qual teste confirmou a mudança?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Da aula 02 em diante: salve os três arquivos em ZIP, abra o Registro de testes no portal, informe nome e turma e baixe o TXT. Entregue ZIP e TXT ao professor na aula. Não há envio nem salvamento automático.

Referências com links

W3C WAI: rótulos dos controles (em inglês)

W3C WAI: notificações e erros (em inglês)

MDN: uso de FormData (em inglês)

MDN: atributo disabled (em inglês)

MDN: setCustomValidity (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

Glossário para consultar

Termo	Papel no projeto
id	Identifica o elemento e permite associação com label.
name	Identifica o dado coletado.
type	Define o comportamento de um input.
value	Representa o valor do controle.
submit	Evento ligado à submissão do formulário.
FormData	Coleta os pares de dados dos controles participantes.
textContent	Insere um valor como texto no DOM.
setCustomValidity	Define ou limpa uma mensagem de validade personalizada.
hidden / disabled	Apresentação / participação na interação, validação e coleta.
HTTPS	Proteção do tráfego em trânsito.

Limite do protótipo

O portal é estático. Resumos e exemplos de requisição permanecem na página. Uma aplicação real precisaria de serviço de recebimento e de conferência dos dados no servidor.

Referências gerais

Base anterior: portal do Prof. Ábner Lucas

MDN: seu primeiro formulário

MDN: tipos de input (em inglês)

W3C WAI: rótulos dos controles (em inglês)

W3C WAI: agrupamento de controles (em inglês)

MDN: validação de formulários

MDN: estilização de formulários (em inglês)

MDN: fundamentos de CSS Grid

MDN: evento submit (em inglês)

MDN: uso de FormData (em inglês)

MDN: setCustomValidity (em inglês)

W3C WAI: notificações e erros (em inglês)

MDN: envio de dados (em inglês)

MDN: elemento button (em inglês)

MDN: atributo disabled (em inglês)

GitHub: funcionamento do Pages

GitHub: configuração da publicação

MDN: textContent (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

MDN: controles nativos de formulário (em inglês)

MDN: HTTPS (em inglês)

Referências verificadas em 22/10/2026. A versão em idioma estrangeiro está indicada no título.